

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG

Naskah Praktikum 1

Database Performance Tuning

Jumat, 18 September 2026

Oleh:

13524059 - Raymond Jonathan Dwi Putra Julianto

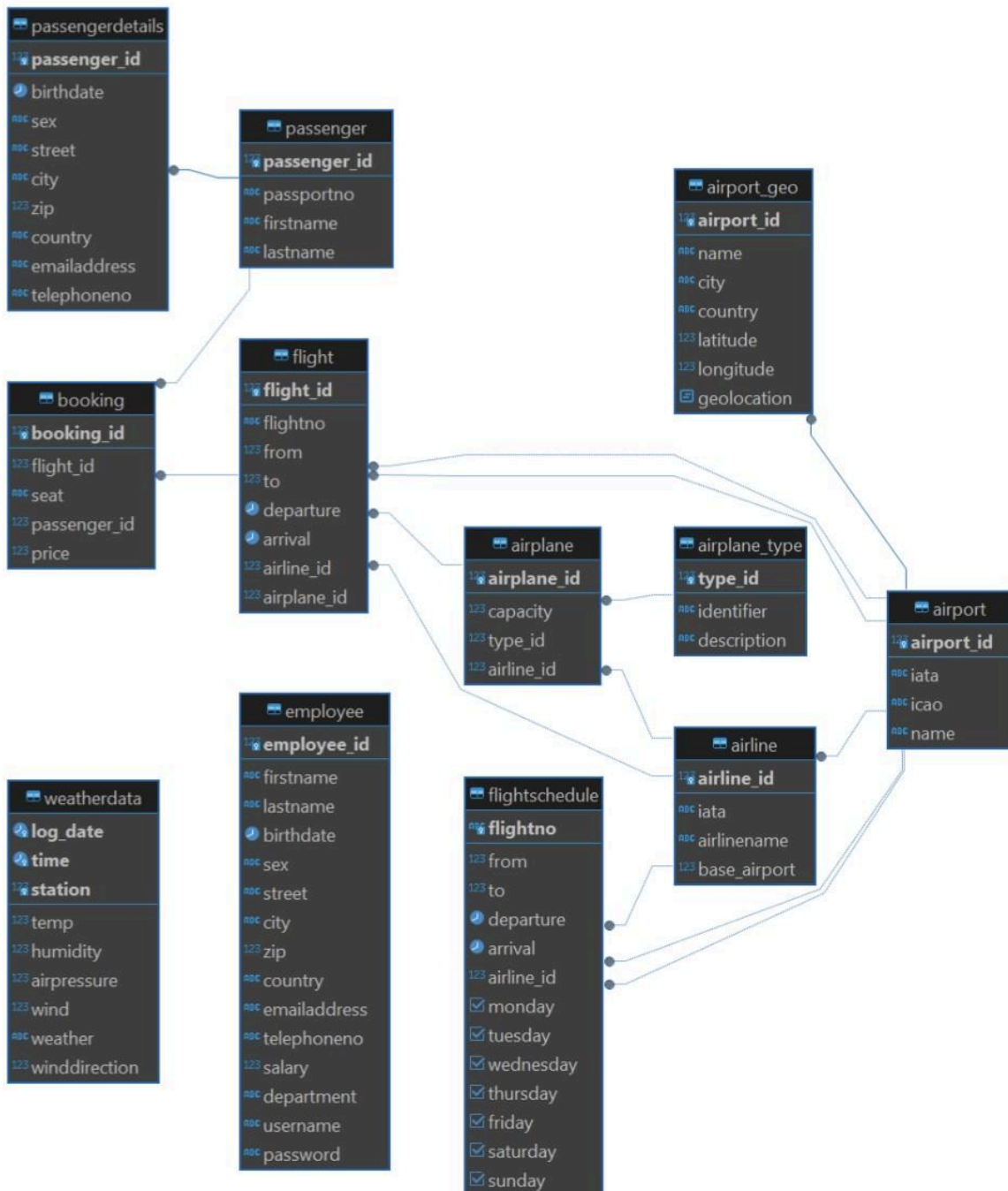
13523060 - Steven Tan

**IF3140 - Sistem Basis Data
2026**

Petunjuk

1. Kerjakan setiap soal praktikum ini dengan baik
2. Query pengujian pada praktikum ini adalah query dengan makna semantik yang sama dengan query awal. Silakan dibuat sendiri jika tidak diberikan
3. Nama database: **airport**
4. **Dilarang Menyontek**
5. **docker exec praktikum-db-1 pg_dump -U postgres -d airport -h localhost -p 5432 > [DATABASE_FILE_EXPORTED].sql**

AirportDB



1. Performance Tuning Theory

Jawab pertanyaan berikut ini dengan baik. Seluruh *query* tidak perlu dijalankan pada basis data karena data tidak tersedia. Namun, setiap *query* yang dipertanyakan harus ditulis secara **SINGKAT**, **LENGKAP**, dan **TEPAT**. Jawaban panjang tidak akan menambah nilai.

- a. Dalam sistem AirportDB, admin sering menampilkan daftar booking yang berisi booking_id, flightno, passenger firstname, passenger lastname, seat, dan price. Data booking terus bertambah sehingga query tersebut semakin sering melakukan join antara tabel booking, passenger, dan flight. Apa metode performance tuning yang paling tepat untuk kasus tersebut dan trade-off apa yang ditimbulkan?

Denormalization antara tabel booking, passenger dan flight. Trade off adalah setiap kali ada query ke hanya salah satu tabel booking, passenger, atau flight juga harus diambil atribut yang tidak relevan.

- b. Laporan pendapatan penerbangan sering digunakan untuk menampilkan total price booking per airline dan per bulan. Jumlah data pada tabel booking terus meningkat sehingga proses agregasi dan join membutuhkan waktu semakin lama. Apa metode performance tuning yang paling tepat untuk meningkatkan performa laporan tersebut dan trade-off apa yang ditimbulkan?

Metode performance yang dibuat adalah dengan membuat sebuah materialized view yang dimana membuat data terkait total price booking per airline dan per bulan

- c. Database AirportDB mendapatkan penambahan data yang sangat signifikan. Apabila query berikut sering dijalankan setiap hari, metode performance tuning apa yang paling tepat digunakan? Sertakan atribut yang terlibat dalam implementasi performance tuning. Alasan tidak perlu dituliskan.

Frequent Query	Jawaban
SELECT * FROM passenger WHERE passportno = {passport};	Index b+ tree tuning pada atribut passportno dari tabel passenger
SELECT * FROM airport WHERE iata = {iata_code};	Index b+ tree tuning pada atribut iata dari tabel airport
SELECT * FROM booking WHERE passenger_id = {passenger_id} AND flight_id = {flight_id};	Index bitmap tuning pada atribut passenger_id and flight_id

- d. Tabel passenger menyimpan informasi identitas penumpang berupa passportno, firstname, dan lastname, sedangkan informasi detail seperti street, city, zip, country, emailaddress, dan telephonenumber juga disimpan pada tabel yang sama. Sistem memiliki fitur pencarian/login penumpang yang hanya membutuhkan passenger_id, passportno, firstname, dan lastname. Apa metode performance tuning yang paling tepat untuk kasus tersebut dan trade-off apa yang ditimbulkan?

Metode yang cocok adalah dengan menggunakan vertical splitting pada Tabel passenger yang dimana membuat tabel baru yang bernama passenger_identity yang berisikan passenger_id, passportno, firstname, dan lastname dan tetap membuat tabel baru yang berisikan sama seperti tabel passenger dengan nama passenger_detail. Namun trade off dari ini adalah perlu adanya konsistensi data pada setiap tabel yang ada pada tabel satu dengan tabel yang lainnya

2. Pakai Apa? Pakai Nanya!

Bibi Tri telah mengumpulkan tiga pertanyaan SQL yang paling sering dijalankan di basis data miliknya. Beliau ingin kamu mengoptimasi ketiga kueri dengan memakai teknik *indexing* yang sesuai.

Untuk tiap pertanyaan, terapkan teknik *indexing* yang sesuai (B-Tree/Hash). Tulis “Indexing tidak cocok” jika *query* tidak cocok dioptimasi dengan *indexing*. Tampilkan keluaran EXPLAIN ANALYZE sebelum dan sesudah optimasi dan beri penjelasan justifikasinya.

```
-- Query 1
SELECT * FROM airport WHERE name LIKE '%Intl%';

-- Query 2
SELECT * FROM employee WHERE country = 'Poland';

-- Query 3
SELECT flight_id, flightno, departure FROM flight
WHERE "from" = 101 ORDER BY departure ASC LIMIT 5;
```

Query 1	SELECT * FROM airport WHERE name LIKE '%Intl%';
Solusi Indexing SQL	[SQL Command/"Index tidak cocok"] Index tidak cocok CREATE INDEX idx_name ON airport (name);
SS Hasil EXPLAIN ANALYZE Sebelum Manipulasi	<pre>----- Seq Scan on airport (cost=0.00..12.50 rows=89 width=27) (actual time=0.031..0.308 rows=95 loops=1) Filter: ((name)::text ~ '%Intl% '::text) Rows Removed by Filter: 505 Planning Time: 0.251 ms Execution Time: 0.343 ms 5 rows)</pre>
SS Hasil EXPLAIN ANALYZE Setelah Manipulasi	<pre>QUERY PLAN ----- Seq Scan on airport (cost=0.00..12.50 rows=89 width=27) (actual time=0.025..0.301 rows=95 loops=1) Filter: ((name)::text ~ '%Intl% '::text) Rows Removed by Filter: 505 Planning Time: 0.883 ms Execution Time: 0.335 ms (5 rows)</pre>
Justifikasi	Tulis dalam poin: Analisis Kebutuhan Kueri, kueri mencoba pattern match name yang yang ada Intl di tengah. Justifikasi Index (jelaskan mekanismenya secara singkat), tidak ada

	index yang cocok karena awalan name bisa apa aja jadi kemungkinan besar bakal ngescan seluruh tabel juga. Jadi baik btree maupun hash hanya akan menambah workload. 3. Justifikasi Kolom (jika dan hanya jika pakai index) 4. Analisis EXPLAIN ANALYZE (jika dan hanya jika pakai index)
--	--

Query 2	SELECT * FROM employee WHERE country = 'Poland';
Solusi Indexing SQL	[SQL Command/"Index tidak cocok"] CREATE INDEX idx_country_employee ON employee USING HASH (country);
SS Hasil EXPLAIN ANALYZE Sebelum Manipulasi	<pre> ----- Seq Scan on employee (cost=0.00..37.50 rows=23 width=160) (actual time=0.109..0.960 rows= 3 loops=1) Filter: ((country)::text = 'Poland'::text) Rows Removed by Filter: 977 Planning Time: 1.715 ms Execution Time: 0.995 ms 5 rows) </pre>
SS Hasil EXPLAIN ANALYZE Setelah Manipulasi	<pre> QUERY PLAN ----- Bitmap Heap Scan on employee (cost=4.18..30.07 rows=23 width=160) (actual time=0.083..0.1 26 rows=23 loops=1) Recheck Cond: ((country)::text = 'Poland'::text) Heap Blocks: exact=14 -> Bitmap Index Scan on idx_country_employee (cost=0.00..4.17 rows=23 width=0) (actual time=0.062..0.062 rows=23 loops=1) Index Cond: ((country)::text = 'Poland'::text) Planning Time: 0.599 ms Execution Time: 0.186 ms (7 rows) </pre>
Justifikasi	Tulis dalam poin: 1. Analisis Kebutuhan Kueri, kueri membutuhkan employee yang atribut countrynya exact match dengan 'Poland' 2. Justifikasi Index (jelaskan mekanismenya secara singkat), hash sangat cocok disini karena pencocokan merupakan equality atau exact match sehingga untuk hash hanya perlu satu kali akses untuk mendapatkan bucket pointer yang menampung index pointer ke record tersebut. 3. Justifikasi Kolom (jika dan hanya jika pakai index), karena atribut yang dicocokkan adalah country jadi country yang diindekskan 4. Analisis EXPLAIN ANALYZE (jika dan hanya jika pakai index), bisa dilihat baik execution time maupun planning berkurang cukup banyak, bisa dilihat perbedaan cost dari seq scan ke bitmap index scan.

Query 3	SELECT flight_id, flightno, departure FROM flight WHERE "from" = 101 ORDER BY departure ASC LIMIT 5;
Solusi Indexing	[SQL Command/"Index tidak cocok"] CREATE INDEX idx_from_flight ON flight USING HASH ("from");

SQL	
SS Hasil EXPLAIN ANALYZE Sebelum Manipulasi	<pre> ----- Limit (cost=149.30..149.31 rows=5 width=19) (actual time=2.831..2.836 rows=5 loops=1) -> Sort (cost=149.30..149.34 rows=18 width=19) (actual time=2.828..2.831 rows=5 loops=1) Sort Key: departure Sort Method: top-N heapsort Memory: 25kB -> Seq Scan on flight (cost=0.00..149.00 rows=18 width=19) (actual time=0.196..2.695 rows=18 loops=1) Filter: ("from" = 101) Rows Removed by Filter: 6782 Planning Time: 0.999 ms Execution Time: 2.904 ms (9 rows) </pre>
SS Hasil EXPLAIN ANALYZE Setelah Manipulasi	<pre> QUERY PLAN ----- Limit (cost=44.66..44.68 rows=5 width=19) (actual time=0.149..0.153 rows=5 loops=1) -> Sort (cost=44.66..44.71 rows=18 width=19) (actual time=0.147..0.149 rows=5 loops=1) Sort Key: departure Sort Method: top-N heapsort Memory: 25kB -> Bitmap Heap Scan on flight (cost=4.14..44.36 rows=18 width=19) (actual time=0.062..0.108 rows=18 loops=1) Recheck Cond: ("from" = 101) Heap Blocks: exact=18 -> Bitmap Index Scan on idx_from_flight (cost=0.00..4.13 rows=18 width=0) (actual time=0.038..0.039 rows=18 loops=1) Index Cond: ("from" = 101) Planning Time: 0.383 ms Execution Time: 0.284 ms (11 rows) </pre>
Justifikasi	Tulis dalam poin: 1. Analisis Kebutuhan Kueri, kueri membutuhkan flight_id, flightno, dan departture dari tabel flight dari tuple yang from = 101 dan terurut berdasarkan departure. 2. Justifikasi Index (jelaskan mekanismenya secara singkat), index hash membantu kueri untuk mencari tuple yang from = 101 3. Justifikasi Kolom (jika dan hanya jika pakai index), from dipilih karena atribut from yang perlu dicocokkan secara exact 4. Analisis EXPLAIN ANALYZE (jika dan hanya jika pakai index), Dengan index, waktu eksekusi jauh menurun baik dari planning time maupun execution time, terutama bisa dilihat perubahanny dari seq scan ke bitmap index scan

3. Duh, Anak Magang!

Terbang Terus Pantang Mundur Airlines, maskapai bertarif murah dengan slogan "Murah, Asal Sabar", menyimpan semua data penumpangnya dalam satu tabel bernama **passengerdetails**. Tabel ini dibuat oleh seorang anak magang yang kurang mementingkan efisiensi.

Tim analis data rutin diminta membuat laporan tren demografi penumpang. Masalahnya, query sederhana untuk mengambil data penumpang inti pun berjalan sangat lambat. Padahal untuk laporan awal, yang benar-benar

mereka butuhkan hanya **passenger_id**, **birthdate**, **sex**, dan **country**. Kolom **street**, **city**, **zip**, **emailaddress**, dan **telephoneno** nyaris tidak pernah disentuh.

Pak Henzy, manajer tim DBA menugaskan kalian (para praktikan) untuk melakukan performance tuning **TANPA MENGUBAH TABEL AWAL** (membuat tabel baru diperbolehkan).

Gunakan perintah explain dan timing untuk mendapatkan variabel analisis! EXPLAIN digunakan untuk melakukan analisis perbandingan cost sebelum dan sesudah melakukan performance tuning. Jika data terlalu panjang, gunakan \q untuk melihat hasil dari timing.

Pada bagian ini, buatlah query untuk mendapatkan data yang sering diperlukan tim analisis!

Query	SELECT * FROM passengerdetails;
SS Hasil Query	
<pre>airport=# SELECT ag.country, SUM(b.price) AS total_revenue, AVG(SUM(b.price)) OVER () AS avg_revenue FROM booking b INNER JOIN flight f ON b.flight_id = f.flight_id INNER JOIN airport a ON f."from" = a.airport_id INNER JOIN airport_geo ag ON a.airport_id = ag.airport_id WHERE EXTRACT(YEAR FROM f.departure) = 2025 AND EXTRACT(MONTH FROM f.departure) BETWEEN 2 AND 5 GROUP BY ag.country ORDER BY SUM(b.price) DESC; Time: 29.261 ms airport=#</pre>	

country	total_revenue	avg_revenue
Indonesia	2323662.69	328173.160425531915
Germany	1352421.22	328173.160425531915
United States	1228406.91	328173.160425531915
Japan	863057.70	328173.160425531915
China	732207.95	328173.160425531915
India	665664.03	328173.160425531915
Italy	584541.41	328173.160425531915
United Kingdom	518111.15	328173.160425531915
Spain	487995.65	328173.160425531915
France	486685.04	328173.160425531915
Brazil	394362.67	328173.160425531915
South Africa	359892.68	328173.160425531915
New Zealand	317397.72	328173.160425531915
Norway	295455.85	328173.160425531915
Sweden	291504.41	328173.160425531915
Austria	277210.35	328173.160425531915
Thailand	271960.31	328173.160425531915
United Arab Emirates	264949.69	328173.160425531915
Poland	258873.04	328173.160425531915
Malaysia	240172.28	328173.160425531915
Australia	225912.84	328173.160425531915
Hong Kong	211857.10	328173.160425531915
Egypt	208345.51	328173.160425531915
Vietnam	202300.04	328173.160425531915
Kenya	196022.20	328173.160425531915
Russia	190194.09	328173.160425531915
Turkey	172568.15	328173.160425531915
Canada	149872.78	328173.160425531915
Philippines	144232.94	328173.160425531915
Switzerland	141127.38	328173.160425531915
Singapore	136779.34	328173.160425531915
Denmark	131592.17	328173.160425531915
Taiwan	131359.80	328173.160425531915
Ireland	129204.46	328173.160425531915
Finland	106117.42	328173.160425531915
Portugal	105790.37	328173.160425531915
Netherlands	85554.56	328173.160425531915
South Korea	81274.62	328173.160425531915
Mexico	80256.76	328173.160425531915
Czech Republic	72017.42	328173.160425531915

Penjelasan Permasalahan dan Jenis *Schema Tuning* yang dilakukan

Masalah yang terjadi adalah dimana atribut yang tidak diperlukan juga ikut dalam proses pencarian data pada query yang terjadi sehingga

memperngaruhi pada berjalannya query. Solusi schema tuning adalah dengan menggunakan vertical splitting yang dimana membuat tabel baru yang berisikan **passenger_id**, **birthdate**, **sex**, dan **country** dengan nama **passengerdetails_identity**.

Saran Perbaikan dan Justifikasinya

Dibuatkan vertical splitting yang dimana membuat tabel baru yang berisikan **passenger_id**, **birthdate**, **sex**, dan **country** dengan nama **passengerdetails_identity** untuk kebutuhan lainnya. Dan diberikan indexing pada masing masing tabel yang ada agar proses pencarian tuple yang sesuai dapat lebih cepat

Query
untuk
Melakukan
Perbaikan

```
Create table passengerdetails_identity AS
SElect passenger_id, birthdate, sex, country
From passengerdetails;

CREATE INDEX idx_from_passengerdetails_identity ON
passengerdetails_identity(passenger_id);

ALTER table passengerdetails_identity ADD
CONSTRAINT
passengerdetails_identity_passenger_id_fkey
FOREIGN KEY (passenger_id) REFERENCES
passenger(passenger_id) ON DELETE CASCADE;

Create table passengerdetails_additional AS
SElect passenger_id, street, city, zip, emailaddress,
telephoneno
From passengerdetails;

ALTER table passengerdetails_additional ADD
CONSTRAINT
passengerdetails_additional_passenger_id_fkey
FOREIGN KEY (passenger_id) REFERENCES
passenger(passenger_id) ON DELETE CASCADE;

CREATE index idx_passengerdetails_additional ON
passengerdetails_additional(passenger_id);
```

SS Query Perbaikan

```
airport=# Create table passengerdetails_identity AS
Select passenger_id, birthdate, sex, country
From passengerdetails;
SELECT 7000
Time: 12.885 ms
airport=# SELECT COUNT(*) FROM passengerdetails;
count
-----
7000
(1 row)

Time: 2.701 ms
```

```
airport=# CREATE INDEX idx_from_passengerdetails_identity ON passengerdetails_identity(passenger_id);
CREATE INDEX
Time: 11.716 ms
```

```
Time: 0.796 ms
airport=# ALTER table passengerdetails_identity ADD CONSTRAINT passengerdetails_identity_passenger_id_fkey FOREIGN KEY (passenger_id) REFERENCES passenger(passenger_id) ON DELETE CASCADE;
ALTER TABLE
Time: 0.357 ms
airport=#
```

```
airport=# Create table passengerdetails_additional AS
Select passenger_id, street, city, zip, emailaddress, telephonenumber
From passengerdetails;
SELECT 7000
Time: 11.687 ms
airport=#
```

```
Time: 11.687 ms
airport=# ALTER table passengerdetails_additional ADD CONSTRAINT passengerdetails_additional_passenger_id_fkey FOREIGN KEY (passenger_id) REFERENCES passenger(passenger_id) ON DELETE CASCADE;
ALTER TABLE
Time: 8.421 ms
airport=#
```

```
airport=# CREATE index idx_passengerdetails_additional ON passengerdetails_additional(passenger_id);
CREATE INDEX
Time: 12.088 ms
airport=#
```

Query
Testing
Setelah
Perbaikan

```
SELECT *
FROM passengerdetails_identity;

SELECT *
FROM passengerdetails_additional;
```

SS Query Testing setelah Perbaikan

passenger_id	birthdate	sex	country
1	1970-04-08	F	Indonesia
2	2006-02-25	F	Germany
3	1953-05-12	M	France
4	1957-11-21	M	Japan
5	2011-05-05	M	United States
6	1950-07-08	F	Indonesia
7	1973-04-26	M	Italy
8	1986-08-04	F	United States
9	2008-03-02	F	Norway
10	1964-11-07	M	Brazil
11	1958-02-02	M	Germany
12	1954-07-05	M	Australia
13	2006-02-05	F	Sweden
14	1964-08-17	F	Japan
15	1969-06-25	F	United States
16	2005-04-25	F	Indonesia
17	1988-09-23	F	Kenya
18	1982-08-24	F	China
19	1978-07-03	M	China
20	1983-05-18	F	France
21	1949-07-14	M	Vietnam
22	2004-09-10	M	Indonesia
23	1985-04-01	F	Japan
24	1987-01-15	F	Poland
25	1981-10-22	F	Germany
26	2008-05-23	F	United States
27	2011-12-03	M	Indonesia
28	1982-05-17	F	Germany
29	1967-01-27	F	Norway
30	1970-11-09	M	France
31	1997-09-15	M	United States
32	2015-05-25	F	Vietnam
33	1962-02-02	M	United States
34	1969-08-05	M	Spain
35	1969-02-06	M	Thailand
36	1962-04-23	F	Indonesia
37	1979-12-05	M	Germany
38	1986-09-25	F	New Zealand
39	1972-03-17	F	China
40	1959-06-17	M	Kenya

passenger_id	street	city	zip	emailaddress	telephonenumber
1	Jalan Merdeka 201	Palembang	17764	erik.bauer1@webmail.net	+23-476-9656379
2	Main Street 182	Koln	26809	gabriel.jayadi2@corp.example.com	+19-325-5899817
3	Schulgasse 146	Lyon	12412	felix.nakamura3@outlook.com	+67-254-5221844
4	Jalan Merdeka 16	Osaka	23981	rania.oktaviani4@webmail.net	+42-797-4627738
5	Ringstrasse 116	Boston	12225	wulan.nugroho5@corp.example.com	+25-923-2129268
6	Church Road 53	Kupang	11580	ika.ivanov6@mail.com	+95-901-4962256
7	Jalan Sudirman 67	Naples	24836	asep.simon7@outlook.com	+62-940-2044183
8	Jalan Ahmad Yani 41	Boston	16516	kenji.lambert8@student.ac.id	+3-771-2235328
9	Rue de la Paix 138	Bergen	10888	lucia.fischer9@outlook.com	+82-940-8424549
10	Gartenweg 16	Rio de Janeiro	12809	lucia.jensen10@webmail.net	+69-563-1072868
11	High Street 203	Bremen	28765	felix.thomas11@mail.com	+27-158-5539460
12	Jalan Thamrin 69	Adelaide	28606	emma.firmansyah12@outlook.com	+13-713-5285011
13	Schulgasse 126	Gothenburg	23844	yuda.kumar13@yahoo.com	+73-831-8717862
14	Hauptstrasse 140	Osaka	22338	jakub.hartono14@outlook.com	+45-719-3739632
15	Main Street 108	Houston	28260	andi.silva15@outlook.com	+60-462-9422402
16	Schulgasse 144	Ambon	16659	aiko.jones16@corp.example.com	+4-228-3532119
17	Schulgasse 196	Mombasa	28905	hannah.moreau17@webmail.net	+55-788-8934480
18	Jalan Merdeka 8	Chengdu	23646	anggun.kurniawan18@gmail.com	+63-446-6511975
19	Bahnhofstrasse 213	Chengdu	12971	kartika.rossi19@yahoo.com	+42-865-8327773
20	Ringstrasse 31	Toulouse	29309	kenji.santoso20@webmail.net	+47-646-4132215
21	Rue de la Paix 88	Hanoi	31461	diego.oktaviani21@outlook.com	+78-809-4144570
22	Maple Drive 73	Ambon	26338	yuda.lefebvre22@proton.me	+3-458-8159853
23	Church Road 30	Osaka	28551	antonio.rodriguez23@mail.com	+73-140-5496158
24	Calle Mayor 21	Krakow	11530	jean.jensen24@yahoo.com	+25-990-8763649
25	Hauptstrasse 180	Stuttgart	18945	akira.yulianto25@webmail.net	+67-542-3229966
26	Jalan Thamrin 192	Houston	25274	mega.dubois26@outlook.com	+42-310-1888084
27	High Street 58	Pontianak	28510	pierre.ivanov27@student.ac.id	+60-269-3193653
28	Jalan Ahmad Yani 13	Koln	30916	hana.schneider28@outlook.com	+32-830-5288911
29	Lindenweg 151	Bergen	20064	antonio.nguyen29@proton.me	+59-497-6631990
30	Jalan Sudirman 116	Lyon	17155	aisyah.hansen30@yahoo.com	+19-885-8157792
31	Ringstrasse 93	Boston	25099	aisyah.schmidt31@student.ac.id	+90-370-7851996
32	Bahnhofstrasse 152	Hanoi	32319	kenji.cahyadi32@mail.com	+23-441-4322704
33	Ringstrasse 180	Phoenix	29015	ari.simon33@corp.example.com	+74-933-2084808
34	Via Roma 50	Malaga	19744	bayu.kurniawan34@yahoo.com	+27-490-8812076
35	Avenida Central 185	Phuket	18008	kartika.walker35@corp.example.com	+51-549-5344430
36	Hauptstrasse 38	Kupang	27882	gabriel.hansen36@webmail.net	+68-579-1480900
37	Oak Lane 163	Stuttgart	17266	ika.popescu37@student.ac.id	+90-606-8550889
38	Jalan Thamrin 110	Wellington	22825	jakub.iskandar38@student.ac.id	+8-941-6813436
39	High Street 140	Shenzhen	21900	hendra.chen39@yahoo.com	+18-398-9709343
40	Jalan Gatot Subroto 33	Mombasa	16328	rosa.setiawan40@yahoo.com	+52-883-7065915

```

airport=# SELECT *
FROM passengerdetails_additional;
Time: 8.259 ms
airport=# SELECT *
FROM passengerdetails_identity;
Time: 7.094 ms

```

Kesimpulan Akhir dan Alasan

Dari hasil yang terbaru, proses pencarian data ini menghasilkan waktu yang lebih cepat dari pada tabel sebelumnya yang dimana proses dapat memangkas waktu yang ada karena hasil vertical split sehingga atribut yang digunakan yang diambil saja.

4. SkyRevenue

Sebuah perusahaan pengelola bandara menggunakan basis data Airport untuk menampilkan total nilai pemesanan tiket berdasarkan negara bandara keberangkatan pada periode Februari–Mei 2025. Laporan tersebut juga menampilkan rata-rata total nilai pemesanan antarnegara.

Laporan diakses berulang kali oleh banyak pengguna melalui dashboard analisis. Setiap kali dashboard dibuka, sistem menjalankan query berikut:

Query	<pre>SELECT ag.country, SUM(b.price) AS total_revenue, AVG(SUM(b.price)) OVER () AS avg_revenue FROM booking b INNER JOIN flight f ON b.flight_id = f.flight_id INNER JOIN airport a ON f."from" = a.airport_id INNER JOIN airport_geo ag ON a.airport_id = ag.airport_id WHERE EXTRACT(YEAR FROM f.departure) = 2025 AND EXTRACT(MONTH FROM f.departure) BETWEEN 2 AND 5 GROUP BY ag.country ORDER BY SUM(b.price) DESC;</pre>
-------	---

Tugas anda sebagai seorang ahli basis data dengan kemampuan +1000 social credit 📊📈 adalah:

1. Menganalisis permasalahan jika query tersebut dijalankan berulang kali oleh banyak pengguna.
2. Memberikan solusi untuk mengatasi permasalahan tersebut beserta query implementasi dan pengujiannya.

Asumsikan sekarang adalah tahun 2026 dan data untuk periode tersebut sudah final serta tidak akan berubah lagi. Hasil laporan tidak boleh berubah, tetapi query pembacaan boleh diganti untuk menggunakan hasil yang telah disimpan.

SS Hasil Query

```
Index Cond: (airport_id = f."from")  
-> Index Only Scan using airport_pkey on airport a (cost=0.28..0.35 rows=1 width=4) (  
    Index Cond: (airport_id = ag.airport_id)  
    Heap Fetches: 14213  
  
Planning Time: 2.299 ms  
Execution Time: 31.451 ms  
(31 rows)  
  
~  
~  
~  
~  
 (END)
```

Penjelasan Permasalahan
Disini query yang dilakukan lumayan cukup banyak menggunakan operasi join sehingga cukup memakan waktu pada operasi ini. Sehingga apabila dilakukan oleh berulang kali dan digunakan banyak orang akan memakan sangat banyak resource yang ada
Saran Perbaikan dan Justifikasinya
Karena query ini digunakan berulang dan tidak dapat diganti maka opsi yang

bisa diberikan adalah dengan memberikan indexing pada proses join yang ada yang dimana index diberikan pada atribut atribut yang dijoinkan lebih tepatnya pada atribut FK dan juga pada atribut yang dicari agar proses pencarian lebih cepat

Query untuk Melakukan Perbaikan	CREATE INDEX if not exists idx_b_flight_id ON booking(flight_id); CREATE INDEX if not exists idx_from_flight ON flight USING HASH ("from"); CREATE INDEX if not exists idx_f_year ON flight(departure);
--	---

SS Query Perbaikan

```
airport=# CREATE INDEX if not exists idx_b_flight_id ON booking(flight_id);
NOTICE: relation "idx_b_flight_id" already exists, skipping
CREATE INDEX
Time: 0.787 ms
```

```
airport=# CREATE INDEX if not exists idx_from_flight ON flight USING HASH ("from");
NOTICE: relation "idx_from_flight" already exists, skipping
CREATE INDEX
Time: 1.094 ms
```

```
airport=# CREATE INDEX if not exists idx_f_year ON flight(departure);
CREATE INDEX
Time: 12.625 ms
```

Query Pengujian	<pre>SELECT ag.country, SUM(b.price) AS total_revenue, AVG(SUM(b.price)) OVER () AS avg_revenue FROM booking b INNER JOIN flight f ON b.flight_id = f.flight_id INNER JOIN airport a ON f."from" = a.airport_id INNER JOIN airport_geo ag ON a.airport_id = ag.airport_id WHERE EXTRACT(YEAR FROM f.departure) = 2025 AND EXTRACT(MONTH FROM f.departure) BETWEEN 2 AND 5 GROUP BY ag.country ORDER BY SUM(b.price) DESC;</pre>
--------------------	---

SS Query Masukan dan Hasil setelah Perbaikan

```

.....
Sort (cost=252.02..252.04 rows=11 width=72) (actual time=14.935..14.937 rows=47 loops=1)
  Sort Key: (sum(b.price)) DESC
  Sort Method: quicksort  Memory: 27kB
  -> WindowAgg (cost=251.47..251.83 rows=11 width=72) (actual time=14.893..14.900 rows=47 loops=1)
    -> GroupAggregate (cost=251.47..251.69 rows=11 width=40) (actual time=12.998..14.872 rows=47 loops=1)
      Group Key: ag.country
      -> Sort (cost=251.47..251.50 rows=11 width=14) (actual time=12.986..13.391 rows=14213 loops=1)
        Sort Key: ag.country
        Sort Method: quicksort  Memory: 981kB
        -> Nested Loop (cost=0.84..251.28 rows=11 width=14) (actual time=0.153..9.984 rows=14213 loops=1)
          -> Nested Loop (cost=0.55..242.68 rows=1 width=12) (actual time=0.120..5.670 rows=1350 loops=1)
            Join Filter: (f."from" = a.airport_id)
            -> Nested Loop (cost=0.28..242.32 rows=1 width=20) (actual time=0.001..4.175 rows=1350 loops=1)
              -> Seq Scan on flight f (cost=0.00..234.00 rows=1 width=8) (actual time=0.033..2.641 rows=1350 loops=1)
                Filter: ((EXTRACT(month FROM departure) >= '2'::numeric) AND (EXTRACT(month FROM departure) <= '5'::numeric) AND (EXTRACT(year FROM departure) = '2025'::numeric))
                Rows Removed by Filter: 5450
              -> Index Scan using airport_geo_pkey on airport_geo ag (cost=0.28..0.29 rows=1 width=12) (actual time=0.001..0.001 rows=1 loops=1350)
                Index Cond: (airport_id = f."from")
            -> Index Only Scan using airport_pkey on airport a (cost=0.28..0.35 rows=1 width=4) (actual time=0.001..0.001 rows=1 loops=1350)
                Index Cond: (airport_id = ag.airport_id)
                Heap Fetches: 1350
          -> Index Scan using idx_b_flight_id on booking b (cost=0.29..0.48 rows=11 width=10) (actual time=0.001..0.002 rows=11 loops=1350)
                Index Cond: (flight_id = f.flight_id)

Planning Time: 2.500 ms
Execution Time: 15.158 ms
(25 rows)

```

Kesimpulan Akhir dan Alasan

Dari perbaikan sini dilihat bahwa indexing pada masing masing atribut join pada FK dapat memangkas waktu proses pencarian yang ada

5. Wong Tidak Teliti

Mr Boss mengadakan sebuah lomba penerbangan terbanyak oleh seorang penumpang. Pemenangnya akan dihadiahkan tanda tangan Messi 🐐, sehingga jutaan orang sangat mengantisipasi kompetisi ini.

Wong, seorang intern, diberi tanggung jawab untuk membantu tim marketing menyiapkan laporan leaderboard yang akan diperbarui dan dipublikasikan setiap hari (misalnya untuk diposting di media sosial dan dikirim ke tim promosi). Wong dan teman-temannya Cloude 🟠, Chat PPT 🟢, dan Geminini 🔵 berhasil membuat query-nya dengan cepat, lalu langsung dipakai tanpa banyak dipikirkan lebih jauh.

Awalnya semua baik-baik saja. Namun setelah kompetisi berjalan beberapa minggu dan jumlah booking membesar hingga jutaan baris, tim marketing mulai komplain, setiap kali mereka menjalankan laporan harian ini, prosesnya butuh waktu sangat lama bahkan sering membuat mereka terlambat mempublikasikan update leaderboard yang dijanjikan setiap pagi. Berikut query yang mereka gunakan:

```
SELECT
  p.passenger_id,
  CONCAT(p.firstname, ' ', p.lastname) AS fullname,
  COUNT(b.booking_id)      AS total_flights,
  SUM(b.price)              AS total_spend,
  MIN(f.departure)         AS first_flight,
  MAX(f.departure)         AS last_flight
FROM passenger p
JOIN booking b   ON b.passenger_id = p.passenger_id
JOIN flight f    ON f.flight_id = b.flight_id
GROUP BY p.passenger_id, fullname
ORDER BY total_spend DESC;
```

Mr Boss 😡 pun memarahi Wong dan teman-temannya, karena keterlambatan ini membuat tim marketing kesulitan menjaga jadwal publikasi harian. Mereka diminta mencari solusi agar laporan ini bisa dijalankan dengan jauh lebih cepat setiap harinya.

Tugas Anda:

Jelaskan mengapa eksekusi query di atas bisa berjalan lambat.

Berikan solusi menggunakan denormalisasi agar laporan leaderboard harian ini bisa diakses jauh lebih cepat oleh tim marketing.

Catatan Penting: Pastikan solusi yang diberikan tidak mengubah/menghapus data asli pada tabel booking, flight, atau passenger.

Query Masukan	[Query yang ditampilkan di atas tersebut]
------------------	---

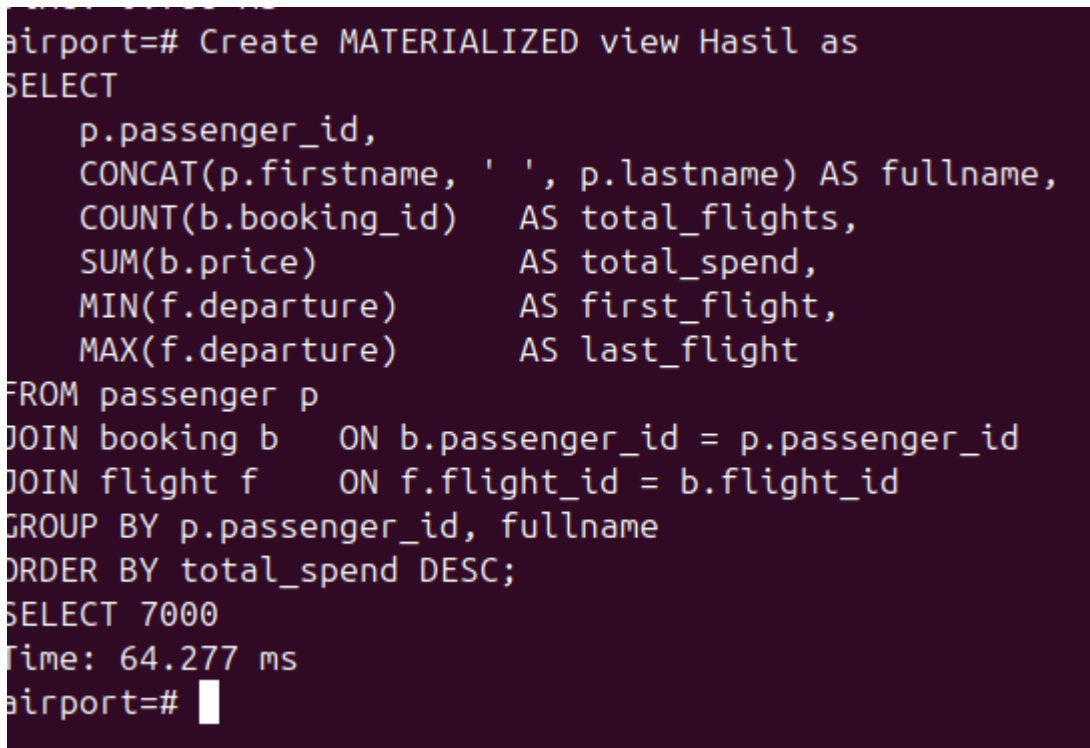
SS Hasil Query Masukan

passenger_id	fullname	total_flights	total_spend	first_flight	last_flight
3332	Erik Fischer	21	27028.24	2025-03-13 17:42:00	2026-08-14 18:37:00
6646	Rahul Zhang	17	26660.50	2025-01-03 06:16:00	2026-05-24 10:22:00
4265	Ricardo Bauer	19	26363.48	2025-01-03 14:48:00	2026-08-29 12:09:00
5741	Carmen Jones	16	26327.23	2025-01-21 21:40:00	2026-06-30 09:22:00
32	Kenji Cahyadi	21	26196.33	2025-02-22 08:39:00	2026-08-16 05:30:00
816	Rina Lesmana	18	25604.23	2025-02-26 08:11:00	2026-04-25 04:34:00
788	Satria Meyer	20	25199.33	2025-01-21 21:40:00	2026-08-20 10:24:00
582	Sofia Martin	19	25124.40	2025-03-12 08:36:00	2026-08-15 14:09:00
5234	Pedro Anggara	22	25112.99	2025-01-14 15:47:00	2026-08-16 05:30:00
678	Bagus Silva	18	25046.42	2025-01-24 19:50:00	2026-08-26 04:55:00
4152	Maya Lambert	19	24991.26	2025-01-09 06:43:00	2026-08-30 13:13:00
3194	Oliver Anggara	19	24907.87	2025-02-13 15:58:00	2026-08-07 09:07:00
1500	Fajar Dubois	19	24766.34	2025-04-29 12:49:00	2026-09-01 18:05:00
5749	Matteo Rodriguez	16	24637.76	2025-01-21 10:37:00	2026-07-02 18:46:00
2766	Matteo Rahardjo	21	24585.03	2025-01-17 07:57:00	2026-08-30 13:13:00
1042	Julia Sorensen	17	24492.76	2025-01-13 06:34:00	2026-08-04 16:35:00
6580	Joko Sanchez	20	24426.90	2025-02-10 08:23:00	2026-08-29 14:11:00
1816	Yusuf Hartono	19	24302.91	2025-01-12 12:58:00	2026-08-19 16:00:00
470	Petra Costa	22	24270.59	2025-01-05 23:11:00	2026-08-22 17:36:00
4734	Julia Wilson	17	24180.63	2025-02-14 08:41:00	2026-08-08 22:08:00
3849	Nicolas Popescu	17	24173.61	2025-01-18 14:08:00	2026-07-02 10:00:00
4358	Dewi Rodriguez	15	24002.41	2025-03-12 14:47:00	2026-07-29 21:36:00
6659	Petra Olsen	22	23880.02	2025-03-01 20:42:00	2026-06-02 22:51:00
5663	Ibrahim Tanuwijaya	14	23485.31	2025-03-28 18:33:00	2026-08-01 13:51:00
3969	Zoe Nguyen	17	23395.78	2025-01-24 17:45:00	2026-08-30 22:08:00
5515	Nikolai Gonzalez	14	23344.67	2025-02-01 06:21:00	2026-09-01 18:05:00
1928	Bambang Yilmaz	18	23268.31	2025-01-07 13:05:00	2026-08-28 17:41:00
2875	Citra Kurniawan	18	23262.04	2025-01-06 17:21:00	2026-08-13 11:53:00
4631	Laura Kumar	17	23250.26	2025-03-21 23:11:00	2026-08-19 11:01:00
1322	Noah Mahendra	17	23227.22	2025-01-15 08:02:00	2026-08-30 17:28:00
4581	Mateo Zulkarnain	18	23086.17	2025-01-09 20:00:00	2026-08-23 21:14:00
5397	Fajar Simon	20	23038.15	2025-01-13 04:17:00	2026-08-10 05:14:00
3209	Laura Walker	17	22992.41	2025-01-06 08:07:00	2026-08-05 13:17:00
6805	Ivan Mahendra	21	22944.58	2025-02-01 04:06:00	2026-09-01 15:18:00
6152	Dedi Muller	17	22925.98	2025-01-08 20:39:00	2026-08-10 17:49:00
3336	Carlos Petrov	20	22915.89	2025-01-11 05:06:00	2026-08-04 16:35:00
1794	Indra Yilmaz	12	22867.40	2025-02-10 11:08:00	2026-08-04 12:30:00
3987	Laura Petrov	19	22786.58	2025-01-02 20:50:00	2026-06-11 11:33:00
5640	Panji Hansen	17	22761.40	2025-01-25 04:55:00	2026-08-29 22:08:00
4052	Lestari Oktaviani	19	22731.43	2025-03-27 04:58:00	2026-08-27 11:15:00

```

airport=# SELECT
  p.passenger_id,
  CONCAT(p.firstname, ' ', p.lastname) AS fullname,
  COUNT(b.booking_id) AS total_flights,
  SUM(b.price) AS total_spend,
  MIN(f.departure) AS first_flight,
  MAX(f.departure) AS last_flight
FROM passenger p
JOIN booking b ON b.passenger_id = p.passenger_id
JOIN flight f ON f.flight_id = b.flight_id
GROUP BY p.passenger_id, fullname
ORDER BY total_spend DESC;
Time: 59.052 ms
airport=#

```

Penjelasan Permasalahan	
Permasalahan yang terjadi adalah terjadi proses agregasi yang banyak yang digunakan untuk menampilkannya pada hasil yang ingin diberikan dan proses ini dilakukan secara berulang ulang pada orang atau sistem yang mengakses dengan query sehingga waktu habis dibuat untuk melakukan proses ini yang sebenarnya orang atau sistem mengakses yang sama	
Saran Perbaikan dan Justifikasinya	
Dibuatkan materialize view yang dimana berisikan isi dari query tersebut dengan akan dibupdate setiap proses perubahan terjadi atau pada cerita ini yang dilakukan pada setiap pagi	
Query untuk Melakukan Perbaikan	<pre>Create MATERIALIZED view Hasil as SELECT p.passenger_id, CONCAT(p.firstname, ' ', p.lastname) AS fullname, COUNT(b.booking_id) AS total_flights, SUM(b.price) AS total_spend, MIN(f.departure) AS first_flight, MAX(f.departure) AS last_flight FROM passenger p JOIN booking b ON b.passenger_id = p.passenger_id JOIN flight f ON f.flight_id = b.flight_id GROUP BY p.passenger_id, fullname ORDER BY total_spend DESC; Refresh MATERIALIZED view HASil;</pre>
	

6. Pilih Pesawat yang Mana

Yeho sedang membantu sebuah maskapai menentukan pesawat yang cocok untuk berbagai penerbangan. Masalahnya, Yeho ternyata cukup sering salah membedakan pesawat kecil dan pesawat besar. Setiap melihat data pesawat, ia harus scroll melewati banyak sekali baris hanya untuk mencari pesawat dengan kapasitas yang sesuai.

Karena mulai lelah scrolling dan takut suatu hari memilih pesawat berkapasitas 100 penumpang untuk penerbangan yang membutuhkan 300 kursi, Yeho ingin mengatur data pesawat menjadi dua kelompok berdasarkan kapasitasnya.

Bantulah Yeho mengusulkan solusi yang sesuai agar data pesawat dengan kapasitas kurang dari 200 penumpang dapat dikelola secara terpisah dari pesawat dengan kapasitas 200 penumpang atau lebih. Tuliskan query yang diperlukan untuk menerapkan solusi tersebut.

Note: Pastikan agar apapun itu strategi yang dilakukan, tabel-tabel awal pada basis data tidak hilang.

Saran Perbaikan dan Justifikasi	
Dibuat dengan schema tuning horizontal split yang kapasitas yang kurang dari 200 akan dibuatkan skemanya tersendiri dan kapasitas dengan lebih dari atau sama dengan dibuatkan tabel tersendiri sehingga kebutuhan dari masing masing dapat dilakukan pada tabel masing masing dan tidak perlu melihat keduanya dan memangkas waktu untuk pencarian yang ada	
Query	<pre>CREATE TABLE airplane_over_two_hundred (LIKE airplane INCLUDING ALL); INSERT INTO airplane_over_two_hundred SELECT * FROM airplane WHERE capacity >= 200; CREATE TABLE airplane_under_two_hundred (LIKE airplane INCLUDING ALL);</pre>

	<pre>INSERT INTO airplane_under_two_hundred SELECT * FROM airplane WHERE capacity < 200;</pre>
SS Hasil Query	
<pre>airport=# CREATE TABLE airplane_over_two_hundred (LIKE airplane INCLUDING ALL); INSERT INTO airplane_two_hundred SELECT * FROM airplane WHERE capacity >= 200; CREATE TABLE airplane_under_two_hundred (LIKE airplane INCLUDING ALL); INSERT INTO airplane_two_hundred SELECT * FROM airplane WHERE capacity < 200; CREATE TABLE Time: 10.173 ms ERROR: relation "airplane_two_hundred" does not exist LINE 1: INSERT INTO airplane_two_hundred ^ Time: 0.223 ms CREATE TABLE Time: 2.867 ms ERROR: relation "airplane_two_hundred" does not exist LINE 1: INSERT INTO airplane_two_hundred ^ Time: 0.190 ms airport=# INSERT INTO airplane_over_two_hundred SELECT * FROM airplane WHERE capacity >= 200; INSERT 0 250 Time: 8.813 ms airport=# INSERT INTO airplane_under_two_hundred SELECT * FROM airplane WHERE capacity < 200; INSERT 0 270 Time: 7.225 ms</pre>	

Pembagian Kerja

NIM	Pembagian Kerja
13524059	1b,1d,3,5,4
13524060	1a,1c,2,4,6